fn match_per_group_predicate

Michael Shoemate (@Shoeboxam)

November 12, 2026

> This proof resides in **"contrib"** because it has not completed the vetting process.

Proves soundness of `match_per_group_predicate` in `mod.rs` at commit `f5bb719` (outdated[1]).

1 Hoare Triple

Precondition

Compiler-verified

Types matching pseudocode.

Caller-verified

None

Pseudocode

```
def match_per_group_predicate(
    enumeration: Expr,
    partition_by: Vec[Expr],
    identifier: Expr,
    threshold: u32,
) -> Optional[Bound]:
    range = int_range(lit(0), len(), 1, DataType.Int64)
    if enumeration == range:
        raise "physical row order is not invariant under unordered identifier metrics"

    trusted = match_permute(enumeration) == range
    if not trusted:
        trusted = match_sort_by_ranks(
            enumeration, range
        )  #
    if not trusted:
        return None

    ids, by = partition(
        lambda expr: expr == identifier, partition_by
    )  #
    if not ids:
        raise "failed to find identifier column in per_group predicate condition"
    return Bound(by=by, per_group=threshold, num_groups=None)  #
```

[1] See new changes with `git diff f5bb719..7d0fdee5b rust/src/transformations/make_stable_lazyframe/truncate/matching/mod.rs`

```
25
26
27 def match_sort_by_ranks(enumeration: Expr, range: Expr) -> bool:
28     if not (
29         isinstance(enumeration, Expr.Function)
30         and enumeration.function == FunctionExpr.ArgSort(False, False)
31         and len(enumeration.input) == 1
32         and isinstance(enumeration.input[0], Expr.SortBy)
33     ):
34         return False
35
36     sort = enumeration.input[0]
37     if sort.expr != range or not sort.by or sort.sort_options.limit is not None:
38         return False
39     *keys, priority = sort.by
40     if match_permute(priority) != range:
41         return False
42
43     for key in keys:  #
44         if not is_rowwise(key):
45             raise "SortBy truncation keys must be rowwise"
46         check_infallible(key, Resize.Ban)
47     return True
```

Postcondition

Theorem 1.1 (Postcondition)**.** If `enumeration` is an enumeration of rows, and `partition_by` includes `identifier`, then returns a `threshold` bound on per-group contribution, when grouped by the non-identifier columns in `partition_by`.

Proof. The matcher accepts only a full trusted uniform permutation of `int_range(0, len)`, or the inverse ranks of that enumeration sorted by rowwise expressions and a final trusted uniform permutation. Line 15 rejects other reorderings.

In the sorted case, each row has a distinct random tie-breaking priority. The sort keys follow rows under input permutations and are infallible, as checked on line 43. Thus the resulting selected rows have the same distribution regardless of physical input order.

By line 21, the window partitions rows by identifier and the additional grouping expressions. Each accepted enumeration assigns distinct ranks $0, \ldots, n-1$ within its partition, so filtering on rank less than `threshold` retains at most that many rows. The bound on line 24 is therefore valid. □